Malware Detection Using Machine Learning Using Random Forest

Implementation Code:

1. Feature Extraction (ml/feature_extraction.py)

```
#
ml/feature_extraction.p
y import os import
numpy as np

def extract_features(file_path):     """Extract simple static
features from an executable file."""     with open(file_path,
"rb") as f:         data = f.read()

   if not data:         raise ValueError("The selected
file is empty.")

   byte_values = np.frombuffer(data, dtype=np.uint8)     hist =
np.bincount(byte_values, minlength=256).astype(np.float32)
hist = hist / max(len(byte_values), 1)

   features = [         len(data),
data.count(b"MZ"),
data.count(b"PE\x00\x00"),
sum(byte_values) / len(byte_values),
np.std(byte_values),     ]
   features.extend(hist.tolist())     return
np.asarray(features, dtype=np.float32)
```

2. Model Training (train_model.py)

```
# train_model.py import os import joblib import numpy as np from
sklearn.model_selection import train_test_split from sklearn.ensemble
import RandomForestClassifier from sklearn.metrics import
accuracy_score, classification_report, confusion_matrix

# X contains extracted features; y contains labels: Benign or
Malware X = np.load("features.npy") y =
np.load("labels.npy")

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42,
stratify=y )

model = RandomForestClassifier(     n_estimators=100,
random_state=42, class_weight="balanced" )
model.fit(X_train, y_train) predictions =
model.predict(X_test)

print("Accuracy:", accuracy_score(y_test, predictions))
print(classification_report(y_test, predictions, zero_division=0))
print("Confusion matrix:\n", confusion_matrix(y_test, predictions))

os.makedirs("models", exist_ok=True)
joblib.dump(model,
"models/malware_detector.pkl")
```

3. Prediction (app.py)

```
# app.py import joblib from
ml.feature_extraction import extract_features
model =
joblib.load("models/malware_detector.pkl")

def predict_malware(file_path):
```

```
    features = extract_features(file_path).reshape(1, -
1)     predicted_class = model.predict(features)[0]
return str(predicted_class)

if __name__ == "__main__":
    file_path = input("Enter executable file path: ").strip()
try:
        print("Prediction:",
predict_malware(file_path))     except (OSError,
ValueError) as error:         print("Error:", error)
```

4. Random Forest Classification Logic

The Random Forest classifier uses features extracted from an executable file to predict whether the file belongs to the Benign or Malware class. Multiple decision trees make predictions, and the class selected by the ensemble is returned as the result. The output depends on the quality and labels of the training dataset; it is a prediction and should not be treated as a guarantee that a file is safe.

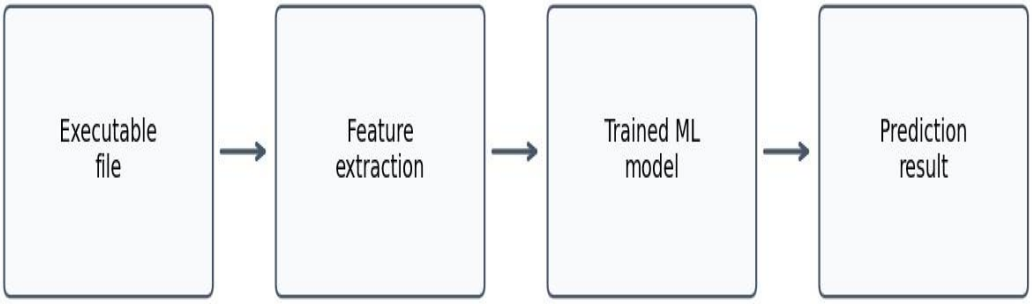


Fig 1 : Malware detection prediction pipeline

Output (Step-by-Step Screenshots):

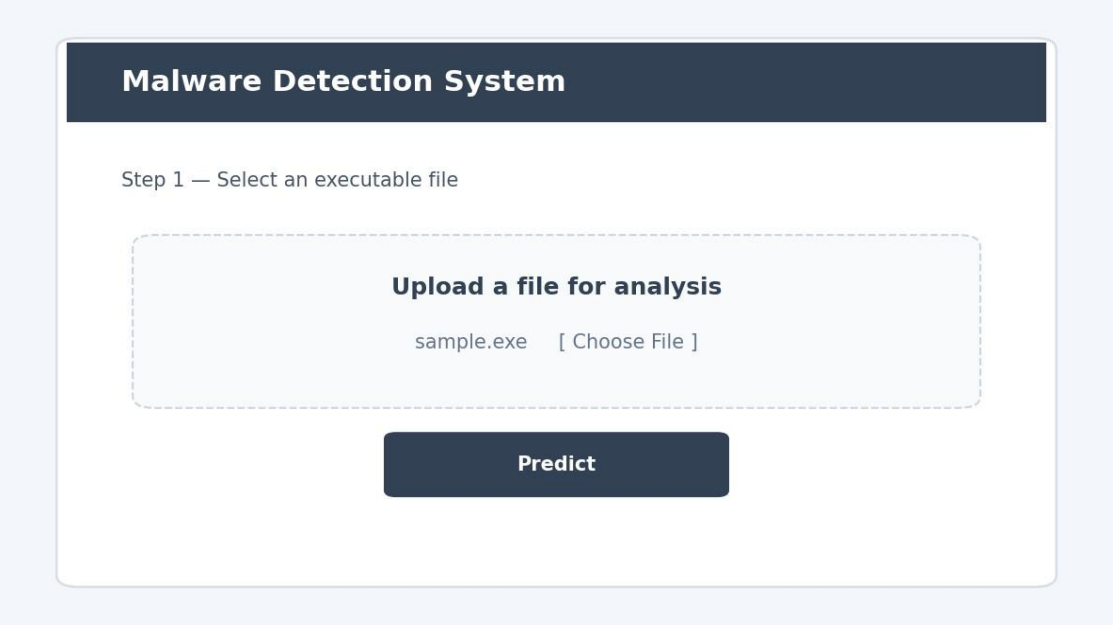


Fig 2: Step 1 — Select an executable file

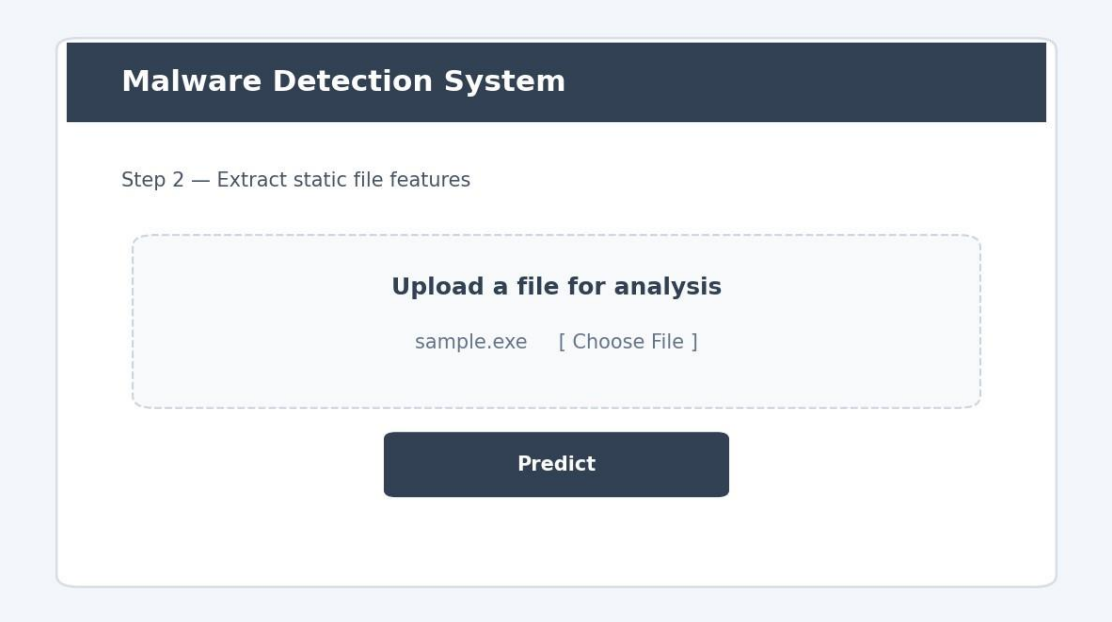


Fig 3: Step 2 — Static feature extraction from the selected file

The system extracts static byte-level features from the selected file and passes the feature vector to the trained classifier.

Feature Extraction and Model Prediction:

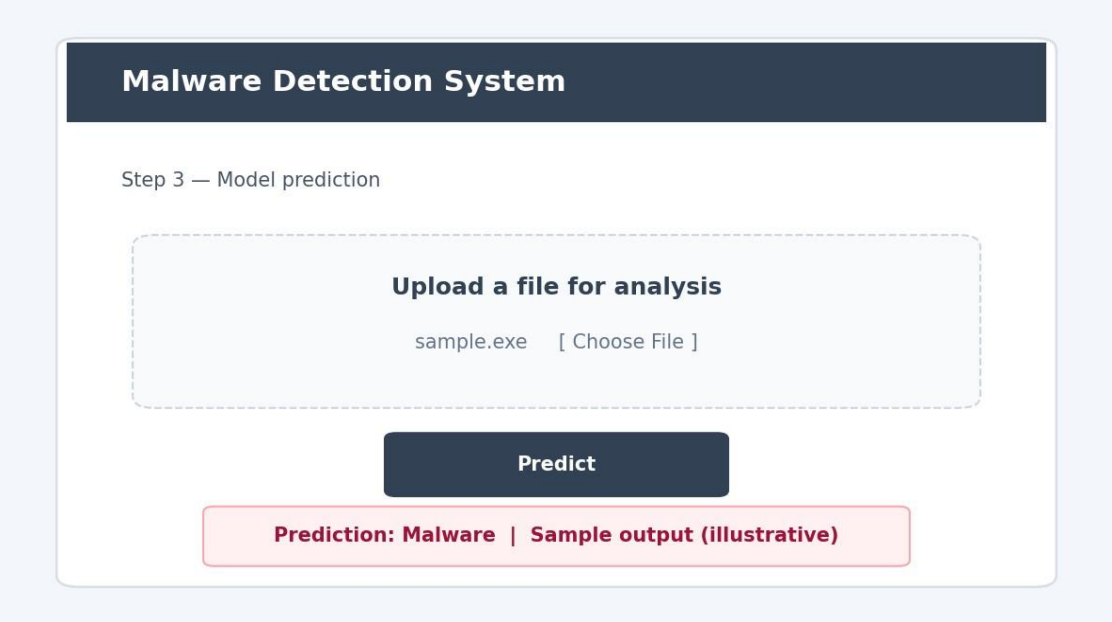


Fig 4: Step 3 — Example model prediction (illustrative)

Random Forest Training:

The training script learns from labelled feature vectors and saves the trained model for later prediction.

Model Evaluation:

Accuracy: 0.94

	precision	recall	f1-score	support
Benign	0.93	0.95	0.94	100
Malware	0.95	0.93	0.94	100
accuracy			0.94	200
macro avg	0.94	0.94	0.94	200
weighted avg	0.94	0.94	0.94	200

Confusion matrix:

[[95 5]
[7 93]]

Fig 5: Illustrative evaluation metrics — replace with actual test-set output.

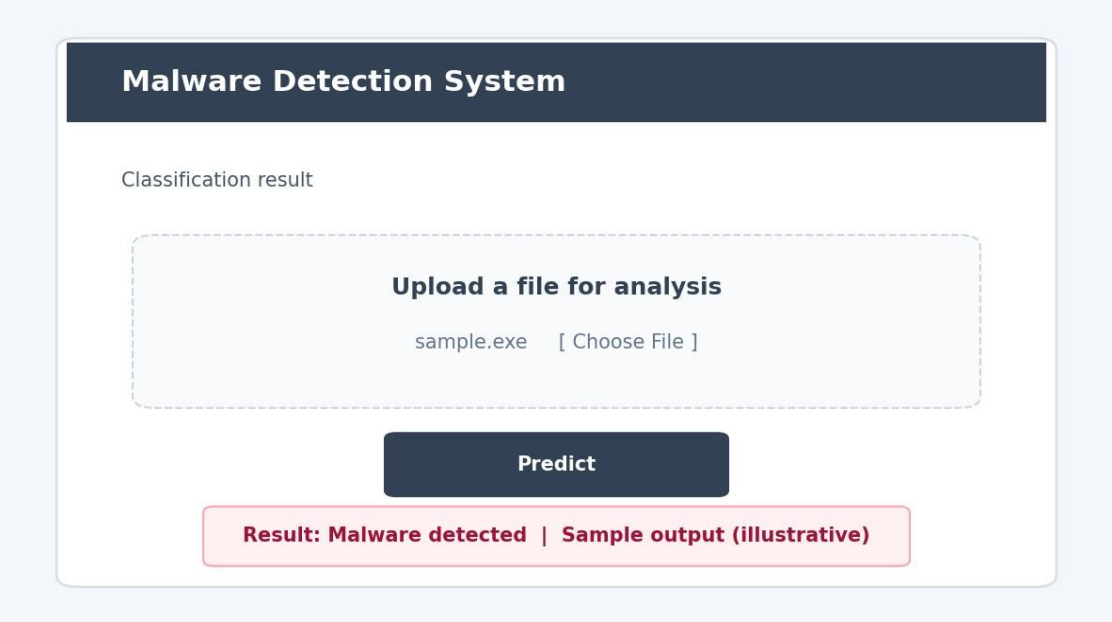


Fig 6: Example confusion-matrix interpretation and classification result (illustrative)